EX NO:11	**DATABASE INTEGRATION IN NODE.JS APPLICATION USING MONGODB**
DATE	15.10.2026
Github Host Link	https://joesuhin10.github.io/EX_11-WEB-TECH/

Aim:

To develop an online toy store website using Node.js and MongoDB by implementing CRUD operations to create, retrieve, update and delete toy information.

Description:

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents called BSON documents.

Node.js is used to handle server-side operations, while the MongoDB Node.js driver connects the application to the database.

The ToyNest online toy store application stores toy details such as toy ID, name, category, brand, stock and price, and manages these records in a MongoDB collection.

Procedure:

1. Install Node.js, npm and MongoDB Community Server.
2. Create a project folder named ToyStoreApp.
3. Open the folder in Visual Studio Code.
4. Initialize the Node.js project using npm init -y.
5. Install the MongoDB driver using npm install mongodb.
6. Create a file named app.js.
7. Import MongoClient and connect to the local server at mongodb://127.0.0.1:27017.
8. Select the ToyStoreDB database and the toys collection.
9. Implement the Create operation using insertMany() to add toy records.
10. Implement the Read operation using find() and findOne() to retrieve toy records.
11. Filter records using the $gt and $in query operators.
12. Implement the Update operation using updateOne() and updateMany().
13. Sort, limit and count records using sort(), limit() and countDocuments().
14. Implement the Delete operation using deleteOne() and deleteMany().
15. Run the program using node app.js and verify the output in the terminal.
16. Create the website using HTML, CSS and JavaScript to display the toy records stored in the database.

Program:

File: app.js

```
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

async function main() {
  try {
    await client.connect();
    console.log("====================================");
    console.log(" CONNECTED TO MONGODB SUCCESSFULLY");
    console.log("====================================");

    const db = client.db("ToyStoreDB");
    const toys = db.collection("toys");
    console.log("Database: ToyStoreDB");
    console.log("Collection: toys");

    // Start fresh so the program can be run again without duplicate records
    await toys.deleteMany({});

    // CREATE
    const insertResult = await toys.insertMany([
      { toyId: 201, name: "LEGO City Fire Station", category: "Building Blocks",
        brand: "LEGO", stock: 25, price: 4500 },
      { toyId: 202, name: "Hot Wheels Mega Track Set", category: "Cars & Vehicles",
        brand: "Hot Wheels", stock: 40, price: 1800 },
      { toyId: 203, name: "Barbie Dreamhouse", category: "Dolls",
        brand: "Barbie", stock: 12, price: 3200 },
      { toyId: 204, name: "Monopoly Classic", category: "Board Games",
        brand: "Hasbro", stock: 30, price: 1200 },
      { toyId: 205, name: "LEGO Technic Racing Car", category: "Building Blocks",
        brand: "LEGO", stock: 18, price: 3800 },
      { toyId: 206, name: "Remote Control Monster Truck", category: "Cars & Vehicles",
        brand: "Zephyr", stock: 8, price: 1900 },
      { toyId: 207, name: "Giant Teddy Bear", category: "Plush Toys",
        brand: "Cuddles", stock: 2, price: 800 },
    ]);
    console.log(`\n${insertResult.insertedCount} Toy Records Inserted Successfully`);

    // READ
    const allToys = await toys.find({}).toArray();
    console.log("\n========== ALL TOYS ==========");
    console.log(allToys);

    const searchToy = await toys.findOne({ name: "Monopoly Classic" });
    console.log("\n========== SEARCH TOY ==========");
    console.log(searchToy);

    const costly = await toys.find({ price: { $gt: 2000 } }).toArray();
    console.log("\n========== PRICE ABOVE 2000 ==========");
    console.log(costly);

    const selected = await toys.find({
      category: { $in: ["Dolls", "Board Games"] },
    }).toArray();
    console.log("\n========== DOLLS AND BOARD GAMES ==========");
    console.log(selected);

    // UPDATE
    const updateResult = await toys.updateOne(
      { toyId: 205 },
      { $set: { price: 3700 } }
    );
    console.log("\nUpdated Records:", updateResult.modifiedCount);

    const updateManyResult = await toys.updateMany(
      { category: "Building Blocks" },
      { $inc: { price: 100 } }
    );
    console.log("Building Blocks Prices Updated:", updateManyResult.modifiedCount);

    const updated = await toys.find({ category: "Building Blocks" }).toArray();
    console.log("\n========== UPDATED TOYS ==========");
    console.log(updated);

    // SORT, LIMIT AND COUNT
```

```
    const sorted = await toys.find({}).sort({ price: -1 }).toArray();
    console.log("\n========== SORTED BY PRICE ==========");
    console.log(sorted);

    const topThree = await toys.find({}).sort({ price: -1 }).limit(3).toArray();
    console.log("\n========== TOP THREE COSTLY TOYS ==========");
    console.log(topThree);

    console.log("\nTotal Number of Toys:", await toys.countDocuments());
    console.log("LEGO Toys:", await toys.countDocuments({ brand: "LEGO" }));

    // DELETE
    const deleteResult = await toys.deleteOne({ toyId: 206 });
    console.log("\nDeleted Records:", deleteResult.deletedCount);

    const deleteManyResult = await toys.deleteMany({ stock: { $lt: 5 } });
    console.log("Toys With Stock Under 5 Deleted:", deleteManyResult.deletedCount);

    const remaining = await toys.find({}).toArray();
    console.log("\n========== REMAINING TOYS ==========");
    console.log(remaining);

    console.log("\nFinal Toy Count:", await toys.countDocuments());
    console.log("\n======================================");
    console.log(" TOY STORE DATABASE OPERATIONS DONE");
    console.log("======================================");
  } catch (error) {
    console.error("Database Error:", error.message);
  } finally {
    await client.close();
    console.log("\nMongoDB Connection Closed");
  }
}

main();
```

File: package.json

```
{
  "name": "toystoreapp",
  "version": "1.0.0",
  "description": "ToyNest online toy store - Node.js and MongoDB CRUD operations",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "mongodb": "^6.10.0"
  }
}
```

Website Source Code:

File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ToyNest - Online Toy Store</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav class="nav">
    <div class="logo">Toy<span>Nest</span></div>
    <ul>
      <li><a href="index.html" class="active">Home</a></li>
      <li><a href="toys.html">Toys</a></li>
      <li><a href="categories.html">Categories</a></li>
      <li><a href="contact.html">Contact</a></li>
      <li><a href="toys.html">Cart <span class="cart-badge" id="cartCount">0</span></a></li>
    </ul>
  </nav>

  <header class="hero">
    <div class="emojis">&#129528; &#128663; &#127922; &#129521; &#127872;</div>
    <h1>Where Every Play Begins</h1>
    <p>Find the perfect toy for every little explorer.</p>
  </header>

  <section class="container">
    <h2>Welcome to ToyNest</h2>
    <p class="sub">Our toy catalogue is built from records stored in a MongoDB database.</p>
    <div class="cards">
      <div class="card stat"><h3><span id="statToys">0</span> Toys</h3><p>Fun picks ready to be
          delivered to your home.</p></div>
      <div class="card stat"><h3><span id="statCategories">0</span> Categories</h3><p>Blocks, cars,
          dolls, board games and more.</p></div>
      <div class="card stat"><h3><span id="statBrands">0</span> Brands</h3><p>Trusted toy makers from
          across the world.</p></div>
    </div>
    <a class="btn home-btn" href="toys.html">Shop Toys</a>
  </section>

  <footer>&copy; 2026 ToyNest Online Toy Store. All rights reserved.</footer>

  <script src="js/data.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

File: toys.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Toys - ToyNest</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav class="nav">
    <div class="logo">Toy<span>Nest</span></div>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="toys.html" class="active">Toys</a></li>
      <li><a href="categories.html">Categories</a></li>
      <li><a href="contact.html">Contact</a></li>
      <li><a href="toys.html">Cart <span class="cart-badge" id="cartCount">0</span></a></li>
    </ul>
  </nav>

  <header class="hero">
    <div class="emojis">&#129528; &#128663; &#127922; &#129521; &#127872;</div>
    <h1>Where Every Play Begins</h1>
    <p>Find the perfect toy for every little explorer.</p>
  </header>

  <section class="container">
```

```
    <h2>Our Toys</h2>
    <p class="sub">Search by toy name or brand, or filter by category.</p>
    <div class="filters">
      <input type="text" id="searchBox" placeholder="Search by name or brand">
      <select id="categoryFilter"><option value="all">All categories</option></select>
    </div>
    <div class="cards" id="toyGrid"></div>
  </section>

  <footer>&copy; 2026 ToyNest Online Toy Store. All rights reserved.</footer>

  <script src="js/data.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

File: categories.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Categories - ToyNest</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav class="nav">
    <div class="logo">Toy<span>Nest</span></div>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="toys.html">Toys</a></li>
      <li><a href="categories.html" class="active">Categories</a></li>
      <li><a href="contact.html">Contact</a></li>
      <li><a href="toys.html">Cart <span class="cart-badge" id="cartCount">0</span></a></li>
    </ul>
  </nav>

  <header class="hero">
    <div class="emojis">&#129528; &#128663; &#127922; &#129521; &#127872;</div>
    <h1>Where Every Play Begins</h1>
    <p>Find the perfect toy for every little explorer.</p>
  </header>

  <section class="container">
    <h2>Toy Categories</h2>
    <p class="sub">Shop by the kind of play your child loves.</p>
    <div class="cards" id="categoryGrid"></div>
  </section>

  <footer>&copy; 2026 ToyNest Online Toy Store. All rights reserved.</footer>

  <script src="js/data.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

File: contact.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Contact - ToyNest</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav class="nav">
    <div class="logo">Toy<span>Nest</span></div>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="toys.html">Toys</a></li>
      <li><a href="categories.html">Categories</a></li>
      <li><a href="contact.html" class="active">Contact</a></li>
      <li><a href="toys.html">Cart <span class="cart-badge" id="cartCount">0</span></a></li>
    </ul>
  </nav>

  <header class="hero">
```

Ex No:11| Database Integration in Node.js Application Using MongoDB

```
    <div class="emojis">&#129528; &#128663; &#127922; &#129521; &#127872;</div>
    <h1>Where Every Play Begins</h1>
    <p>Find the perfect toy for every little explorer.</p>
  </header>

  <section class="container">
    <h2>Contact Us</h2>
    <p class="sub">Questions about an order or a toy? Send us a message.</p>
    <div class="contact-wrap">
      <div class="card">
        <form id="contactForm">
          <label for="n">Your name</label>
          <input id="n" type="text" required>
          <label for="e">Email</label>
          <input id="e" type="email" required>
          <label for="m">Message</label>
          <textarea id="m" rows="4" required></textarea>
          <button class="btn" type="submit">Send Message</button>
          <p id="thanks">Thank you! We will reply soon.</p>
        </form>
      </div>
      <div class="card">
        <h3>ToyNest Store</h3>
        <p><b>Email:</b> hello@toynest.in</p>
        <p><b>Phone:</b> +91 98765 43210</p>
        <p><b>Address:</b> Coimbatore, Tamil Nadu</p>
        <p><b>Hours:</b> Mon - Sat, 10 AM to 8 PM</p>
      </div>
    </div>
  </section>

  <footer>&copy; 2026 ToyNest Online Toy Store. All rights reserved.</footer>

  <script src="js/data.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

File: css/style.css

```
:root {
  --navy: #2d2a6e;
  --purple: #5b3fd0;
  --yellow: #ffc93c;
  --coral: #ff6b6b;
  --teal: #2ec4b6;
  --cream: #fff9ee;
  --ink: #2d2a4a;
  --muted: #6b6a85;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: "Poppins", "Segoe UI", Arial, sans-serif;
  background: var(--cream);
  color: var(--ink);
}

/* ---------- Navigation ---------- */
.nav {
  background: var(--navy);
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 32px;
}
.logo { font-size: 1.5rem; font-weight: 700; color: #fff; }
.logo span { color: var(--yellow); }
.nav ul { list-style: none; display: flex; gap: 8px; margin: 0; padding: 0; align-items: center; }
.nav a {
  color: #fff; text-decoration: none; font-size: 0.85rem;
  padding: 7px 14px; border-radius: 20px;
}
.nav a.active { background: var(--yellow); color: var(--navy); font-weight: 600; }
.nav a:hover { background: rgba(255, 255, 255, 0.15); }
.nav a.active:hover { background: var(--yellow); }
.cart-badge {
  background: var(--coral); color: #fff; border-radius: 12px;
  padding: 1px 8px; font-size: 0.75rem; margin-left: 4px;
```

Ex No:11| Database Integration in Node.js Application Using MongoDB

```
}

/* ---------- Banner ---------- */
.hero {
  background: linear-gradient(135deg, var(--navy), var(--purple));
  text-align: center;
  padding: 44px 20px;
}
.hero h1 { margin: 0 0 6px; color: var(--yellow); font-size: 2rem; }
.hero p { margin: 0; color: #e8e6ff; font-size: 0.95rem; }
.hero .emojis { font-size: 1.8rem; letter-spacing: 10px; margin-bottom: 8px; }

/* ---------- Layout ---------- */
.container { max-width: 960px; margin: 0 auto; padding: 36px 20px 50px; }
.container h2 { margin: 0 0 6px; font-size: 1.7rem; color: var(--navy); }
.sub { color: var(--muted); font-size: 0.9rem; margin: 0 0 22px; }

.cards { display: grid; grid-template-columns: repeat(auto-fill, minmax(260px, 1fr)); gap: 18px; }
.card {
  background: #fff; border-radius: 18px; padding: 18px;
  border-top: 5px solid var(--yellow);
  box-shadow: 0 4px 14px rgba(45, 42, 110, 0.10);
}
.card:nth-child(3n + 2) { border-top-color: var(--coral); }
.card:nth-child(3n) { border-top-color: var(--teal); }
.card h3 { margin: 0 0 8px; font-size: 1.05rem; color: var(--navy); }
.card p { margin: 4px 0; font-size: 0.85rem; color: var(--muted); }
.card p b { color: var(--ink); }

.stat h3 { font-size: 1.3rem; }

/* ---------- Toy cards ---------- */
.toy-img {
  background: #fff1c9; border-radius: 14px; height: 90px;
  display: flex; align-items: center; justify-content: center;
  font-size: 3rem; margin-bottom: 12px;
}
.card:nth-child(3n + 2) .toy-img { background: #ffe0e0; }
.card:nth-child(3n) .toy-img { background: #d6f5f1; }
.tag {
  display: inline-block; background: #ece8ff; color: var(--purple);
  font-size: 0.72rem; font-weight: 600; padding: 2px 10px; border-radius: 12px; margin-bottom: 6px;
}
.price { font-size: 1.25rem !important; font-weight: 700; color: var(--coral) !important; margin-top:
    8px !important; }

.btn {
  background: var(--purple); color: #fff; border: none; cursor: pointer;
  padding: 10px 22px; border-radius: 24px; font-family: inherit;
  font-weight: 600; font-size: 0.85rem; text-decoration: none; display: inline-block;
}
.btn:hover { background: var(--navy); }
.toy .btn { width: 100%; margin-top: 6px; }
.home-btn { margin-top: 22px; background: var(--yellow); color: var(--navy); }
.home-btn:hover { background: #ffb800; }

/* ---------- Filters ---------- */
.filters { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; margin-bottom: 22px; }
.filters input, .filters select {
  padding: 11px 14px; border: 2px solid #e6e1f7; border-radius: 12px;
  font-family: inherit; font-size: 0.85rem; background: #fff;
}
.empty { color: var(--muted); }

/* ---------- Categories ---------- */
.cat-icon { font-size: 2rem; margin-bottom: 4px; }

/* ---------- Contact ---------- */
.contact-wrap { display: grid; grid-template-columns: 1.4fr 1fr; gap: 20px; }
.contact-wrap label { display: block; font-size: 0.8rem; font-weight: 600; margin: 10px 0 4px; }
.contact-wrap input, .contact-wrap textarea {
  width: 100%; padding: 10px 12px; border: 2px solid #e6e1f7;
  border-radius: 12px; font-family: inherit; font-size: 0.85rem;
}
.contact-wrap button { margin-top: 14px; }
#thanks { display: none; color: var(--teal); font-weight: 600; margin-top: 10px; font-size: 0.85rem; }

/* ---------- Footer ---------- */
footer { background: var(--navy); color: #d8d5ff; text-align: center; padding: 18px; font-size:
    0.8rem; }

@media (max-width: 640px) {
```

```
  .nav { flex-direction: column; gap: 10px; }
  .filters, .contact-wrap { grid-template-columns: 1fr; }
}
```

File: js/data.js

```
// Copy of the toy records left in MongoDB (ToyStoreDB.toys) after app.js ran
const toys = [
  { toyId: 201, name: "LEGO City Fire Station", category: "Building Blocks",
    brand: "LEGO", stock: 25, price: 4600, icon: "\u{1F9F1}" },
  { toyId: 202, name: "Hot Wheels Mega Track Set", category: "Cars & Vehicles",
    brand: "Hot Wheels", stock: 40, price: 1800, icon: "\u{1F697}" },
  { toyId: 203, name: "Barbie Dreamhouse", category: "Dolls",
    brand: "Barbie", stock: 12, price: 3200, icon: "\u{1F380}" },
  { toyId: 204, name: "Monopoly Classic", category: "Board Games",
    brand: "Hasbro", stock: 30, price: 1200, icon: "\u{1F3B2}" },
  { toyId: 205, name: "LEGO Technic Racing Car", category: "Building Blocks",
    brand: "LEGO", stock: 18, price: 3800, icon: "\u{1F3CE}\uFE0F" },
];
```

File: js/main.js

```
// ---------- Cart counter (kept in the browser) ----------
let cartCount = 0;
try { cartCount = Number(localStorage.getItem("toynestCart")) || 0; } catch (e) {}

function updateCartBadge() {
  const badge = document.getElementById("cartCount");
  if (badge) badge.textContent = cartCount;
}

function addToCart(name) {
  cartCount++;
  try { localStorage.setItem("toynestCart", cartCount); } catch (e) {}
  updateCartBadge();
  alert(name + " added to your cart!");
}

// ---------- Home page statistics ----------
function showStats() {
  const set = (id, value) => {
    const el = document.getElementById(id);
    if (el) el.textContent = value;
  };
  set("statToys", toys.length);
  set("statCategories", new Set(toys.map((t) => t.category)).size);
  set("statBrands", new Set(toys.map((t) => t.brand)).size);
}

// ---------- Toys page ----------
function renderToys() {
  const grid = document.getElementById("toyGrid");
  if (!grid) return;
  const text = document.getElementById("searchBox").value.toLowerCase();
  const cat = document.getElementById("categoryFilter").value;

  const list = toys.filter((t) =>
    (t.name.toLowerCase().includes(text) || t.brand.toLowerCase().includes(text)) &&
    (cat === "all" || t.category === cat)
  );

  grid.innerHTML = list.length
    ? list.map((t) => `
      <div class="card toy">
        <div class="toy-img">${t.icon}</div>
        <h3>${t.name}</h3>
        <span class="tag">${t.category}</span>
        <p><b>Toy ID:</b> ${t.toyId}</p>
        <p><b>Brand:</b> ${t.brand}</p>
        <p><b>In Stock:</b> ${t.stock} pieces</p>
        <p class="price">₹${t.price}</p>
        <button class="btn" onclick="addToCart('${t.name}')">Add to Cart</button>
      </div>`).join("")
    : '<p class="empty">No toys found.</p>';
}

function setupFilters() {
  const select = document.getElementById("categoryFilter");
  if (!select) return;
  [...new Set(toys.map((t) => t.category))].sort().forEach((c) => {
```

Ex No:11| Database Integration in Node.js Application Using MongoDB

```
    select.innerHTML += `<option value="${c}">${c}</option>`;
  });
  document.getElementById("searchBox").addEventListener("input", renderToys);
  select.addEventListener("change", renderToys);
  renderToys();
}

// ---------- Categories page ----------
function renderCategories() {
  const box = document.getElementById("categoryGrid");
  if (!box) return;
  const names = [...new Set(toys.map((t) => t.category))].sort();
  box.innerHTML = names.map((c) => {
    const items = toys.filter((t) => t.category === c);
    return `
      <div class="card cat">
        <div class="cat-icon">${items[0].icon}</div>
        <h3>${c}</h3>
        <p>${items.map((t) => t.name).join(", ")}</p>
      </div>`;
  }).join("");
}

// ---------- Contact page ----------
function setupContact() {
  const form = document.getElementById("contactForm");
  if (!form) return;
  form.addEventListener("submit", (e) => {
    e.preventDefault();
    document.getElementById("thanks").style.display = "block";
    form.reset();
  });
}

updateCartBadge();
showStats();
setupFilters();
renderCategories();
setupContact();
```

Output:

Terminal output of node app.js:

```
PS C:\ToyStoreApp> node app.js
====================================
 CONNECTED TO MONGODB SUCCESSFULLY
====================================
Database: ToyStoreDB
Collection: toys

7 Toy Records Inserted Successfully

========== ALL TOYS ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4500
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3c'),
    toyId: 202,
    name: 'Hot Wheels Mega Track Set',
    category: 'Cars & Vehicles',
    brand: 'Hot Wheels',
    stock: 40,
    price: 1800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3e'),
    toyId: 204,
    name: 'Monopoly Classic',
    category: 'Board Games',
    brand: 'Hasbro',
    stock: 30,
    price: 1200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d40'),
    toyId: 206,
    name: 'Remote Control Monster Truck',
    category: 'Cars & Vehicles',
    brand: 'Zephyr',
    stock: 8,
    price: 1900
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d41'),
    toyId: 207,
    name: 'Giant Teddy Bear',
    category: 'Plush Toys',
    brand: 'Cuddles',
    stock: 2,
    price: 800
  }
]
```

```
========== SEARCH TOY ==========
{
  _id: new ObjectId('6aeca1dd85841b0726f43d3e'),
  toyId: 204,
  name: 'Monopoly Classic',
  category: 'Board Games',
  brand: 'Hasbro',
  stock: 30,
  price: 1200
}

========== PRICE ABOVE 2000 ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4500
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  }
]
```

```
========== DOLLS AND BOARD GAMES ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3e'),
    toyId: 204,
    name: 'Monopoly Classic',
    category: 'Board Games',
    brand: 'Hasbro',
    stock: 30,
    price: 1200
  }
]

Updated Records: 1
Building Blocks Prices Updated: 2

========== UPDATED TOYS ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4600
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  }
]
```

```
========== SORTED BY PRICE ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4600
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d40'),
    toyId: 206,
    name: 'Remote Control Monster Truck',
    category: 'Cars & Vehicles',
    brand: 'Zephyr',
    stock: 8,
    price: 1900
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3c'),
    toyId: 202,
    name: 'Hot Wheels Mega Track Set',
    category: 'Cars & Vehicles',
    brand: 'Hot Wheels',
    stock: 40,
    price: 1800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3e'),
    toyId: 204,
    name: 'Monopoly Classic',
    category: 'Board Games',
    brand: 'Hasbro',
    stock: 30,
    price: 1200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d41'),
    toyId: 207,
    name: 'Giant Teddy Bear',
    category: 'Plush Toys',
    brand: 'Cuddles',
    stock: 2,
    price: 800
  }
]
```

```
========== TOP THREE COSTLY TOYS ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4600
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  }
]

Total Number of Toys: 7
LEGO Toys: 2

Deleted Records: 1
Toys With Stock Under 5 Deleted: 1
```

```
========== REMAINING TOYS ==========
[
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3b'),
    toyId: 201,
    name: 'LEGO City Fire Station',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 25,
    price: 4600
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3c'),
    toyId: 202,
    name: 'Hot Wheels Mega Track Set',
    category: 'Cars & Vehicles',
    brand: 'Hot Wheels',
    stock: 40,
    price: 1800
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3d'),
    toyId: 203,
    name: 'Barbie Dreamhouse',
    category: 'Dolls',
    brand: 'Barbie',
    stock: 12,
    price: 3200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3e'),
    toyId: 204,
    name: 'Monopoly Classic',
    category: 'Board Games',
    brand: 'Hasbro',
    stock: 30,
    price: 1200
  },
  {
    _id: new ObjectId('6aeca1dd85841b0726f43d3f'),
    toyId: 205,
    name: 'LEGO Technic Racing Car',
    category: 'Building Blocks',
    brand: 'LEGO',
    stock: 18,
    price: 3800
  }
]

Final Toy Count: 5

====================================
 TOY STORE DATABASE OPERATIONS DONE
====================================

MongoDB Connection Closed
```

Website output:

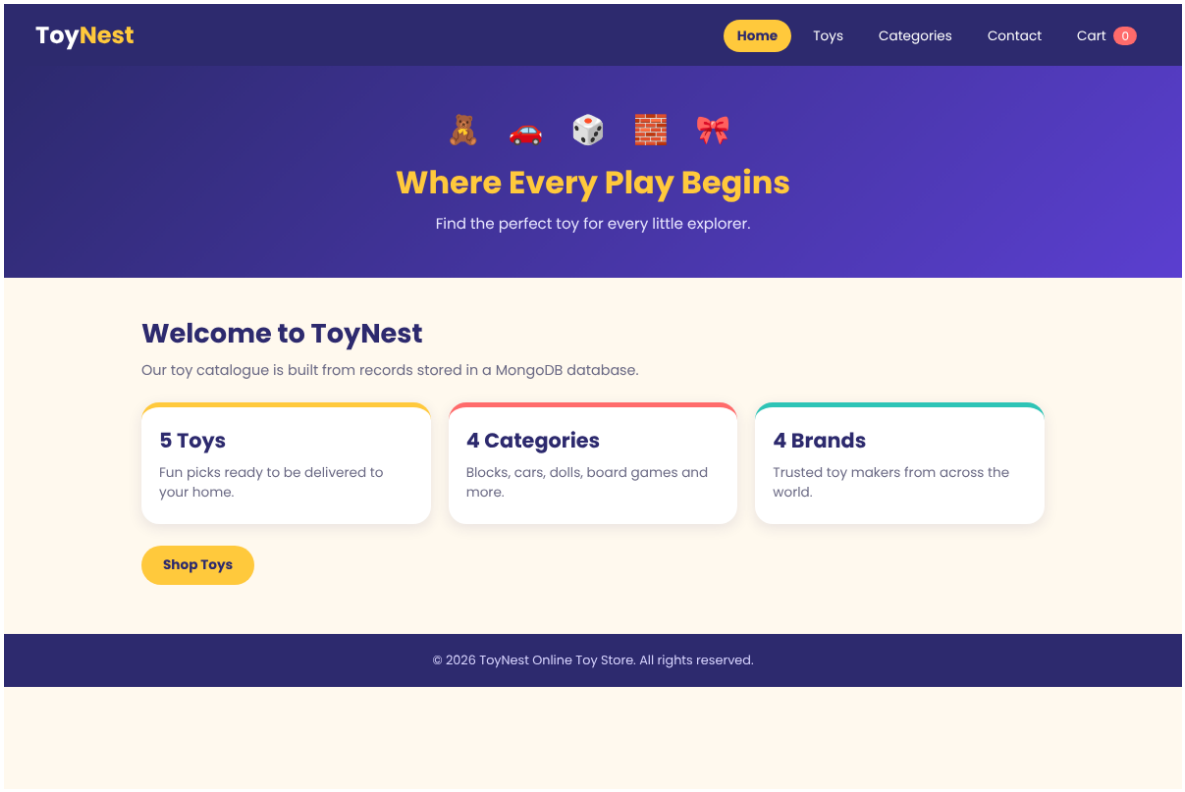


Fig 1: ToyNest home page

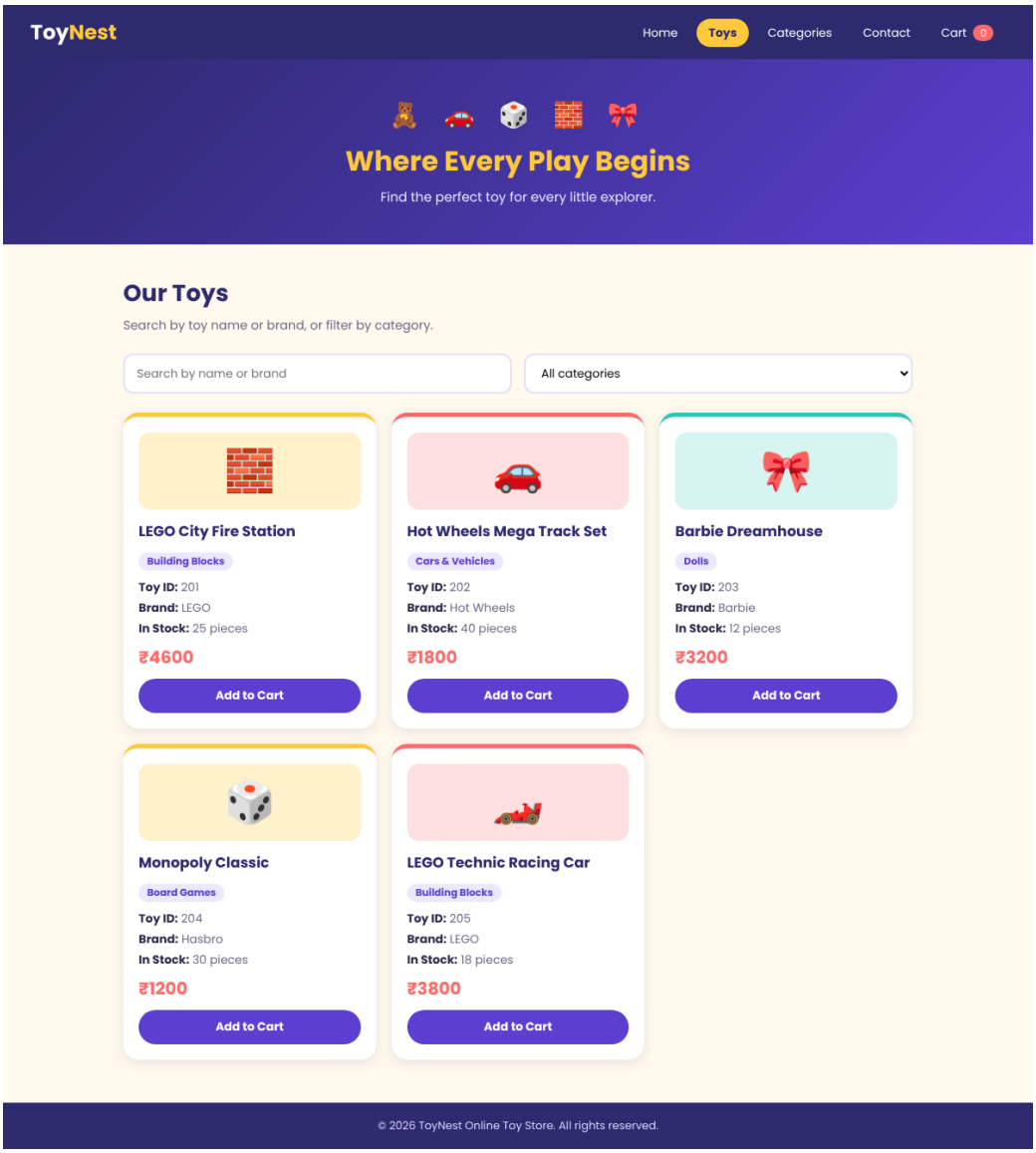


Fig 2: Toys page showing the records left in MongoDB after the program ran

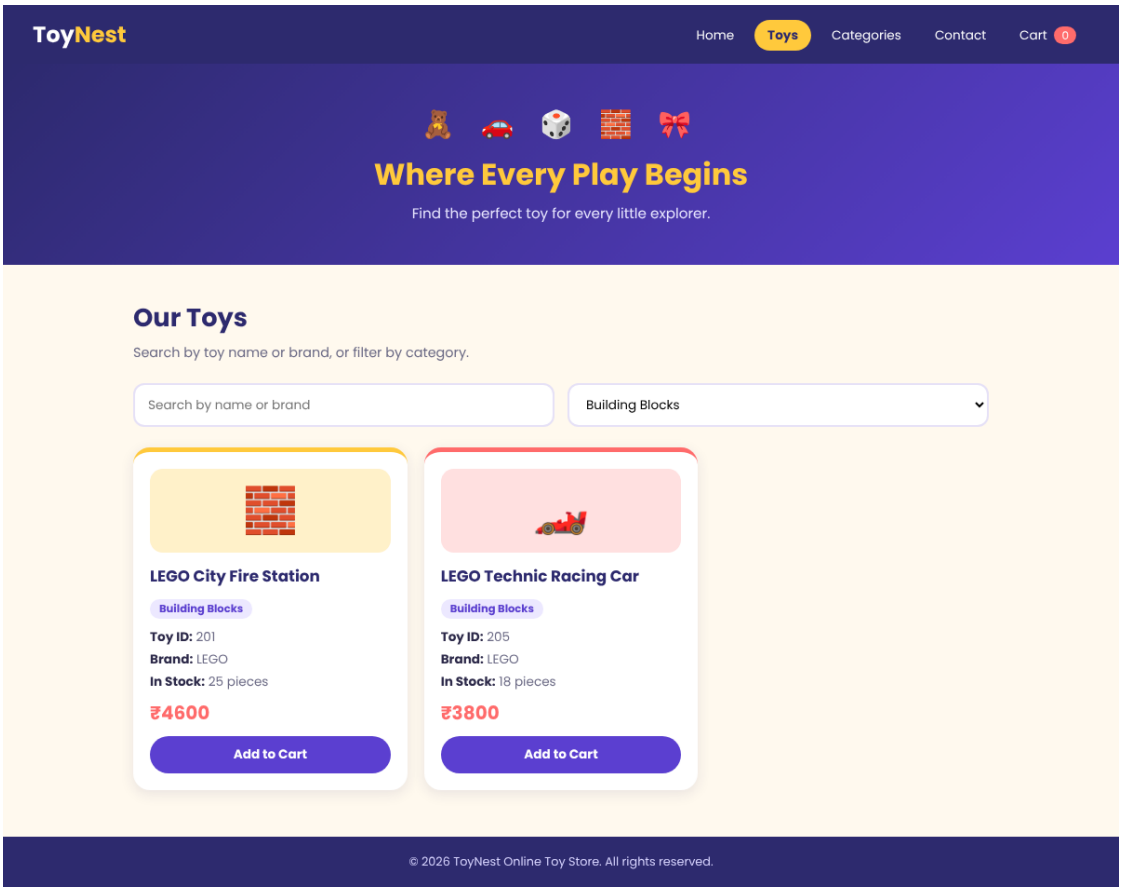


Fig 3: Toys filtered by category, with updated prices

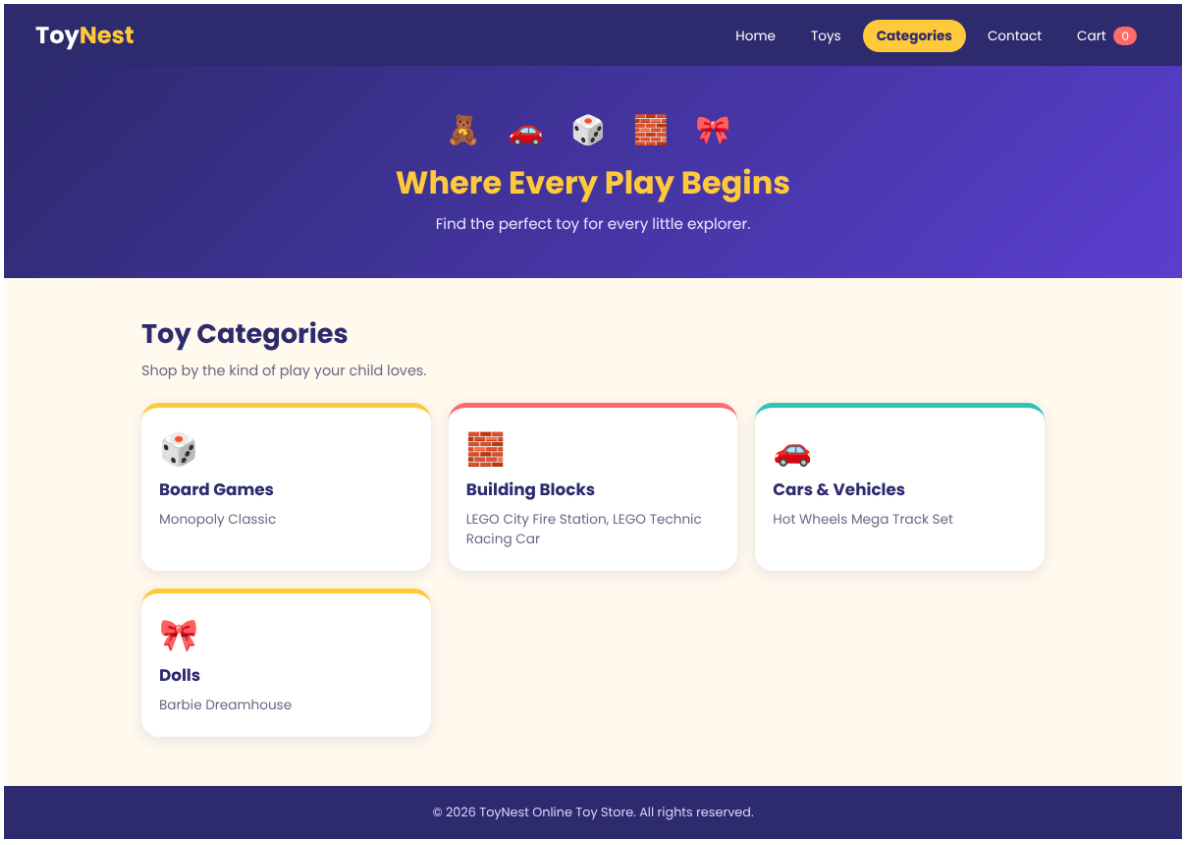


Fig 4: Categories page

Ex No:11| Database Integration in Node.js Application Using MongoDB

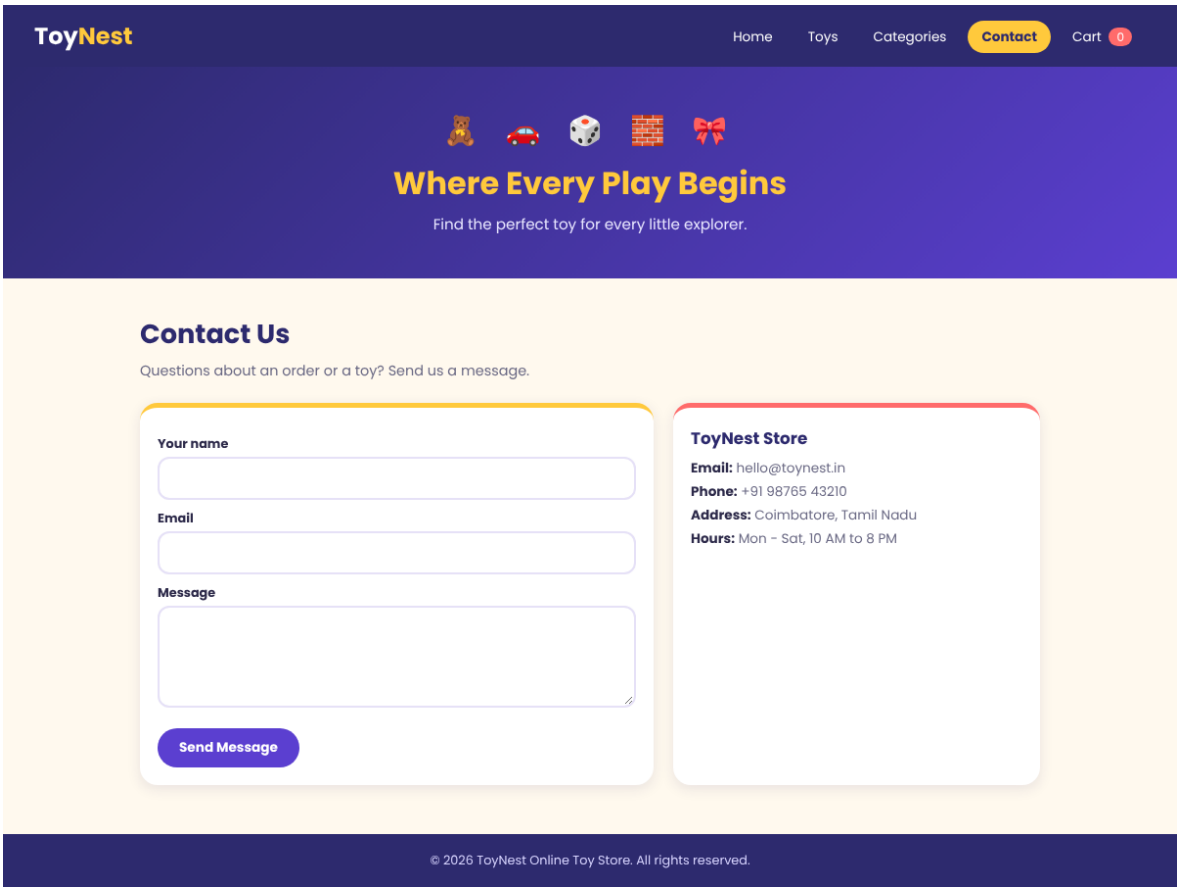


Fig 5: Contact page

Result:

A Node.js application was successfully developed and integrated with MongoDB for the ToyNest Online Toy Store Website. The application established database connectivity and implemented insertion, retrieval, updating, and deletion of toy records successfully. The website displays a copy of the stored records and is hosted at https://joesuhin10.github.io/EX_11-WEB-TECH/